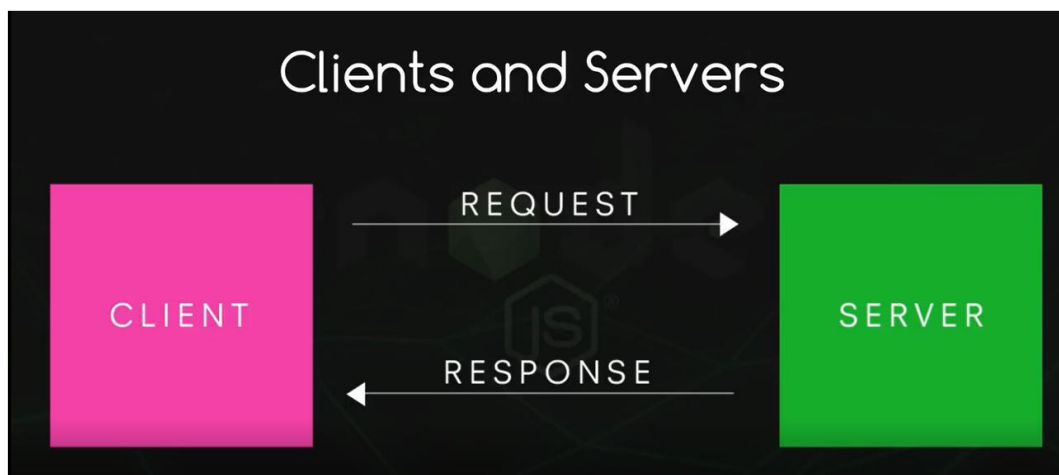


Contenido

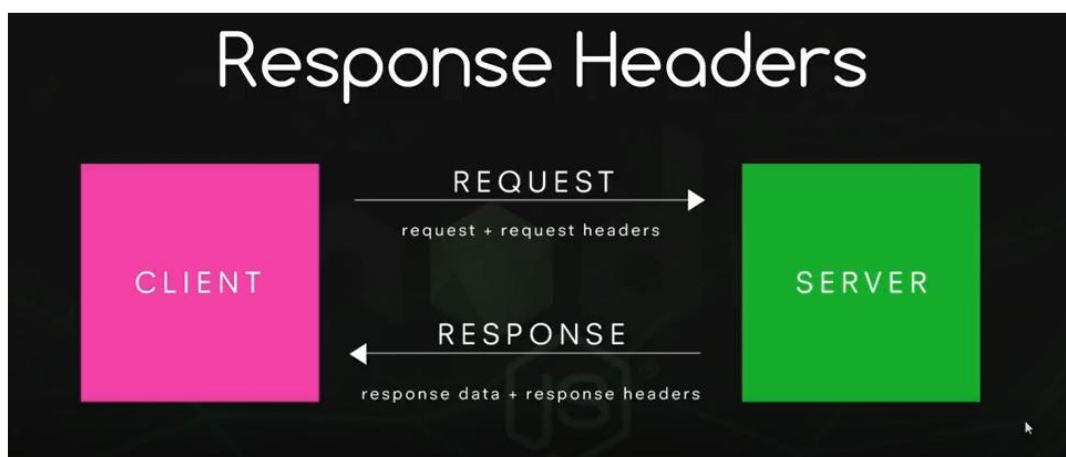
1. Servidores	1
2. Servir JSON	3
2.1. Entrega de JSON al cliente	3
2.2. Para trabajar con un archivo JSON:	4
2.2.1. Lectura de un archivo JSON	4
2.2.2. Escribir en un archivo JSON	6

1. Servidores

En primer lugar, recordemos que un cliente que realiza una solicitud a un servidor, que a su vez envía una respuesta, se comunica con este mediante protocolos: reglas predefinidas para establecer comunicaciones a través de sockets, que son un tipo de canal de comunicación que utiliza puertos TCP o UDP. Cada protocolo TCP/IP de la capa de aplicación realiza una función: transmisión de información web, transferencia de archivos, correo electrónico, etc.



La información adicional se envía mediante encabezados.



Por ejemplo, en la respuesta, el servidor indica al cliente el tipo de contenido que envía para saber qué hacer con él. Un navegador mostrará la información HTML como una página web, pero no mostrará un archivo de texto o JSON. El estado del envío también aparece en los encabezados de respuesta: 200 si todo es correcto, error 404, etc.

En Node, podemos crear servidores http utilizando el módulo http.

```
const http = require('http');
```

Para crearlo, utilizamos el método `createServer` de este módulo, que recibe como parámetro una función con dos parámetros: uno que almacena la solicitud realizada y el otro, la respuesta.

```
var server = http.createServer(function (req, res) {  
});
```

http.createServer([options][, requestlistener])

[\[src\]](#)

► History

- `options` <Object>
 - `IncomingMessage` <`http.IncomingMessage`> Specifies the `IncomingMessage` class to be used. Useful for extending the original `IncomingMessage`. Default: `IncomingMessage`.
 - `ServerResponse` <`http.ServerResponse`> Specifies the `ServerResponse` class to be used. Useful for extending the original `ServerResponse`. Default: `ServerResponse`.
- `requestListener` <Function>
- Returns: <`http.Server`>

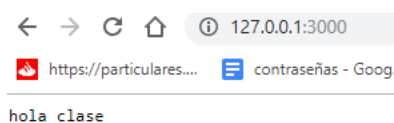
Returns a new instance of `http.Server`.

The `requestListener` is a function which is automatically added to the `'request'` event.

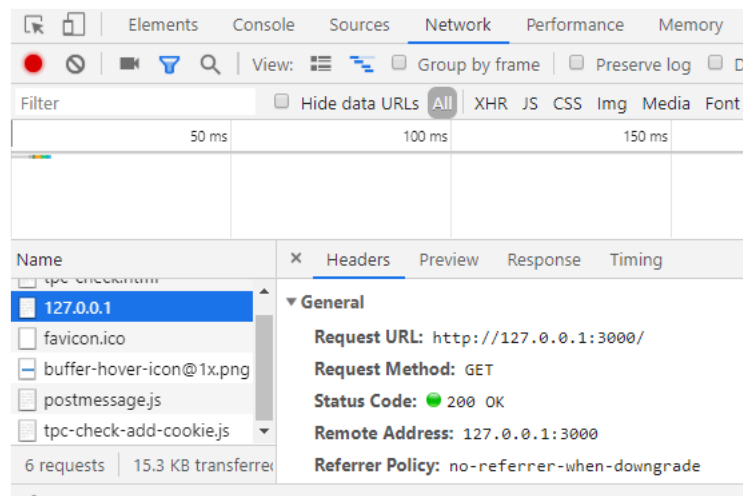
Vamos a especificar el encabezado de respuesta donde indicaremos que se envía texto plano y crearemos dicho texto. También es necesario definir un puerto y la IP. Usaremos 3000 y, el servidor que crearemos ahora será local 127.0.0.1.

```
var server = http.createServer(function (req, res) {  
  res.writeHead(200, { 'content-type': 'text/plain' });  
  res.end('hola clase');  
});  
  
server.listen(3000, '127.0.0.1');
```

Ahora, vamos a un navegador web y escribimos:



Si hacemos clic derecho en la página, podremos ver la solicitud en la sección de red (recarga la página después de abrir el inspector si no te sale a la primera).



2. Servir JSON

2.1. Entrega de JSON al cliente

Recordamos que JSON es una forma de escribir datos con notación JavaScript:

https://www.w3schools.com/js/js_json_intro.asp

<https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/JSON>

Se usa ampliamente para enviar datos entre servidores y clientes. Por esta razón, debemos aprender a enviarlos desde nuestro servidor node.

Lo primero es cambiar el tipo de datos que enviamos. También creamos un objeto JSON simple y podemos pensar en algo como esto:

```
const http = require('http');

var server = http.createServer(function (req, res) {
  res.writeHead(200, { 'content-type': 'application/json' });
  var miObj = {
    nombre: 'pepe',
    trabajo: 'programador',
    edad: 35,
  };
  res.end(miObj);
});

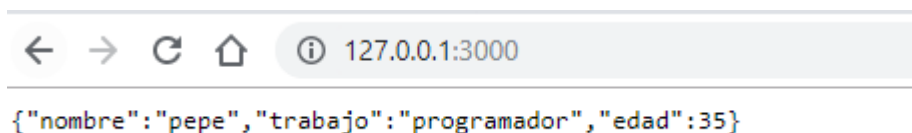
server.listen(3000, '127.0.0.1');
```

Sin embargo, el método final toma una cadena como parámetro, mientras que miObj no. Para solucionar esto, lo convertimos en cadena. Para convertir una cadena a JSON, usamos JSON.stringify(string):

```
const http = require('http');

var server = http.createServer(function (req, res) {
  res.writeHead(200, { 'content-type': 'application/json' });
  var miObj = {
    nombre: 'pepe',
    trabajo: 'programador',
    edad: 35,
  };
  res.end(JSON.stringify(miObj));
});

server.listen(3000, '127.0.0.1');
```



A screenshot of a web browser's address bar and content area. The address bar shows navigation icons (back, forward, refresh, home) and the URL '127.0.0.1:3000'. Below the address bar, the browser displays the JSON response: {"nombre": "pepe", "trabajo": "programador", "edad": 35}.

Una solicitud JSON puede provenir de código JavaScript que la genera y luego muestra los datos de una manera específica en el front-end de una página web. Lo habitual es trabajar con archivos JSON. Podemos combinar los modos asíncrono y síncrono que vimos anteriormente con los métodos que veremos ahora.

2.2. Trabajar con un archivo JSON:

2.2.1. Lectura de un archivo JSON

- **Método 1:** Usar el método require: El método más sencillo para leer un archivo JSON es requerirlo en un archivo node.js mediante el método require().

Sintaxis:

```
const data = require('path/to/file/filename');
```

Ejemplo: Cree un archivo users.json en el mismo directorio que el archivo index.js. Añada los siguientes datos al archivo JSON.

Archivo users.json:

```
[
  {
    "name": "John",
```

```
[
  {
    "age": 21,
    "language": ["JavaScript", "PHP", "Python"]
  },
  {
    "name": "Smith",
    "age": 25,
    "language": ["PHP", "Go", "JavaScript"]
  }
]
```

Ahora, agrega el siguiente código a tu archivo index.js.

Archivo index.js:

```
// Archivo de usuarios requeridos
const users = require('./users');
console.log(users);
```

Ahora, ejecuta el archivo con el comando:

node index.js

- **Método 2:** Usar el módulo fs: También podemos usar el módulo fs de node.js para leer un archivo. El módulo fs devuelve el contenido del archivo en formato de cadena, por lo que debemos convertirlo a formato JSON mediante el método integrado JSON.parse().

Agrega el siguiente código a tu archivo index.js:

Archivo index.js:

```
const fs = require('fs');

// Leer el archivo users.json
fs.readFile('users.json', function (err, data) {
  // Comprobar errores
  if (err) throw err;
  // Convertir a JSON
  const users = JSON.parse(data);
  console.log(users); // Imprimir users
});
```

Ahora ejecute el archivo de nuevo.

2.2.2. Escribir en un archivo JSON

Podemos escribir datos en un archivo JSON usando el módulo fs de node.js. Podemos usar el método writeFile para escribir datos en un archivo.

Sintaxis:

```
s.writeFile("filename", data, callback);
```

Ejemplo: Agregaremos un nuevo usuario al archivo JSON existente, creado en el ejemplo anterior. Esta tarea se completará en tres pasos:

1. Leer el archivo usando uno de los métodos anteriores.
2. Agregar los datos usando el método `push()`.
3. Escribir los nuevos datos en el archivo usando el método `JSON.stringify()` para convertir los datos en una cadena.

```
const fs = require('fs');
// Paso 1: Leer archivo JSON
const users = require('./users');
// Definir el nuevo usuario
let user = {
  name: 'Nuevo usuario',
  age: 30,
  language: ['PHP', 'Go', 'JavaScript'],
};
// Paso 2: Añadir los datos nuevos al objeto users
users.push(user);
// Paso 3: Escribirlo en el archivo
fs.writeFile('users.json', JSON.stringify(users), (err) => {
  // Comprobando errores
  if (err) throw err;
  console.log('Hecho');
});
```

Ejecuta el archivo de nuevo y comprueba que haya cambiado `users.json`.